



CS & IT ENGINEERING



Algorithms

Dynamic Programming

Lecture No.- 02



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Dijkstra's SSSP
Intro to DP

Topics to be Covered



Topic

Topic

Topic

Dynamic Programming

0/1 Knapsack



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

TTTe



Topic : Dynamic Programming

#Q. Consider the following table :

Algorithm	Design Paradigms
(P) Kruskal	(i) Divide and Conquer
(Q) Quick sort	(ii) Greedy
(R) Floyd-Warshall	(iii) Dynamic Programming

Match the algorithms to the design paradigms they are based on.

HW

89%

A (P) $\leftrightarrow$ (ii), (Q) $\leftrightarrow$ (iii), (R) $\leftrightarrow$ (i) ~~X~~

B (P) $\leftrightarrow$ (iii), (Q) $\leftrightarrow$ (i), (R) $\leftrightarrow$ (ii) ~~X~~

C (P) $\leftrightarrow$ (ii), (Q) $\leftrightarrow$ (i), (R) $\leftrightarrow$ (iii)

D (P) $\leftrightarrow$ (i), (Q) $\leftrightarrow$ (ii), (R) $\leftrightarrow$ (iii) ~~X~~

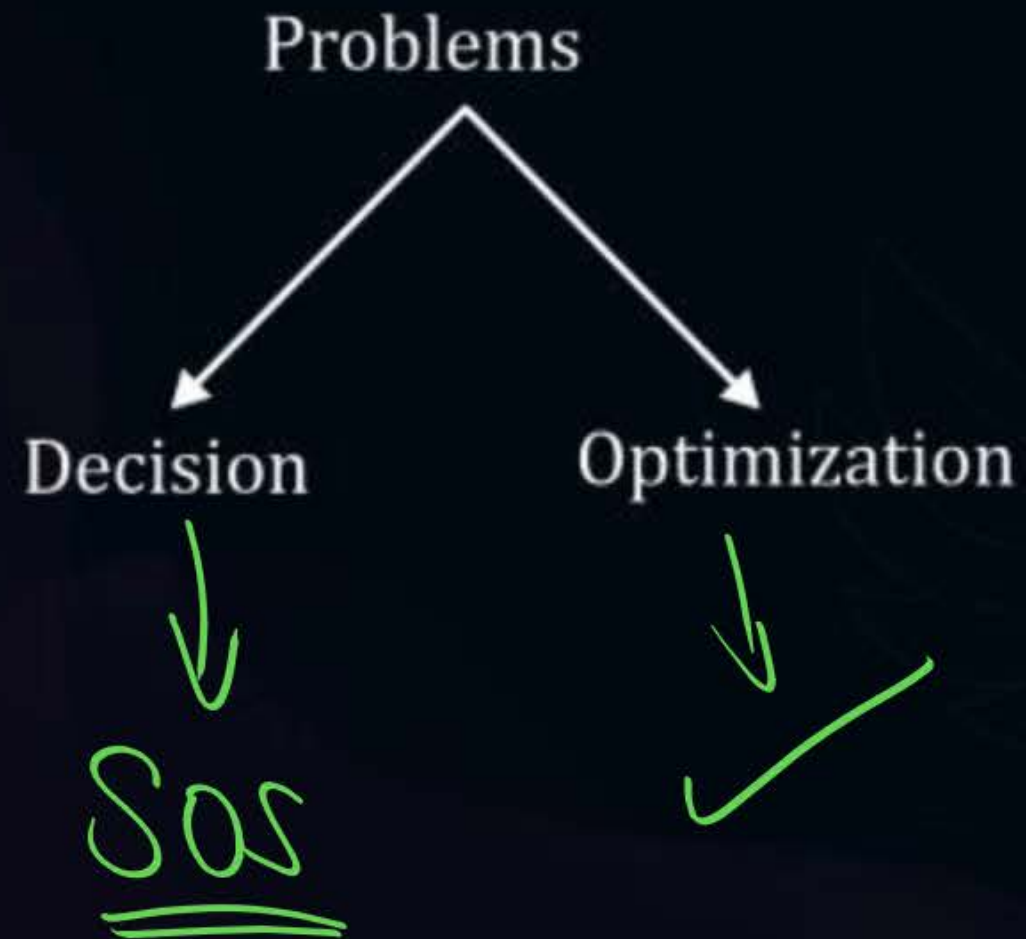


Topic : Dynamic Programming

Introduction to Dynamic Programming:

~~Sorting~~/ Tabulating the values or Results of the sub- Problems.

Storing





Topic : Dynamic Programming

Dynamic Programming (DP) is an algorithm design method used for solving problems, whose solutions are viewed as a result of making a set / sequence of decisions.

- One way of making these decisions is to make them one at a time in a step-wise (sequential) step-by-step manner and never make an erroneous decision. This is true of all problems solvable by Greedy method.)
- For many other problems it is not possible to make step-wise decisions based on local information available at every step, In such a manner that the sequence of decision made is optimal.

DP



Topic : Dynamic Programming

Eg. 0/1 Knapsack



Sometimes greedy appr. Does not give optimal solution to some problems.



Topic : Dynamic Programming

Consider the weights and values of times listed below: Note that there is only one unit of each item.

$M = 11$ (Capacity)

0/1 Knapsack

Item number	Weight (in kgs)	Profit (in Rupees)
1	10	60
2	7	28
3	4	20
4	2	24

Binary Knapsack



Topic : Dynamic Programming

	W	P
01	10	60
02	7	28
03	4	20
04	2	24

$$\rightarrow 60 / 10 = 6 \rightarrow 2 \times x_1 = 0$$

$$\rightarrow 27 / 7 = 4 \rightarrow 4 \times x_2 = 0$$

$$\rightarrow 40 / 4 = 5 \rightarrow 3 \times x_3 = 1$$

$$\rightarrow 24 / 2 = 12 \rightarrow 4 \times x_1 = 1$$

$$M = 11$$

$$11 - 2 = 9$$

$$10 > 9 \rightarrow \text{Skip}$$

$$9 - 4 = 5$$

greedy

Max profit by ~~queued~~ appr. To 0/1 knapsack

$$= \sum_{i=1}^4 p_i x_i$$

$$= 60 \cdot 0 + 28 \times 0 + 20 \times 1$$

$$= 20 + 24 + 0 + 0$$

$$= 44$$



Topic : Dynamic Programming

But optimal profit

when taken only object O_1

= 60

Hence, greedy approach fails to give optimal solution in 0/1 knapsack (may or not give)



Topic : Dynamic Programming

Eg.1. Coin change Problem (DP vs Greedy)

Problem:- Given a set of coin values, construct a sum of money using as few coins as possible.

You can use each coin value any number of times.

Coin Values: $\{C_1, C_2 \dots C_K\}$

Target Money (Sum) : N



Topic : Dynamic Programming

Eg.1. Coins = {1,2,5}, Sum = N = 12

Greedy Method: $\underline{5 + 5 + 2} = 12$

Number of coins required = 3

in this case, Greedy gives optimal ~~assume.~~ ^{solv}



Topic : Dynamic Programming

Eg.1. Coins = {1, 3, 4} , N = 6 (sum)

Greedy approach: 4 + 1 + 1 = 6 (Number of coins = 3)

in general, optimal sol. 3 + 3 = 6 (number of coins = 2)



Topic : Dynamic Programming

In above example , greedy approach failed to give optimal solution.

Hence, greedy approach doesn't guarantee (may or may not give) optimal solution to coins change problem.





Topic : Dynamic Programming

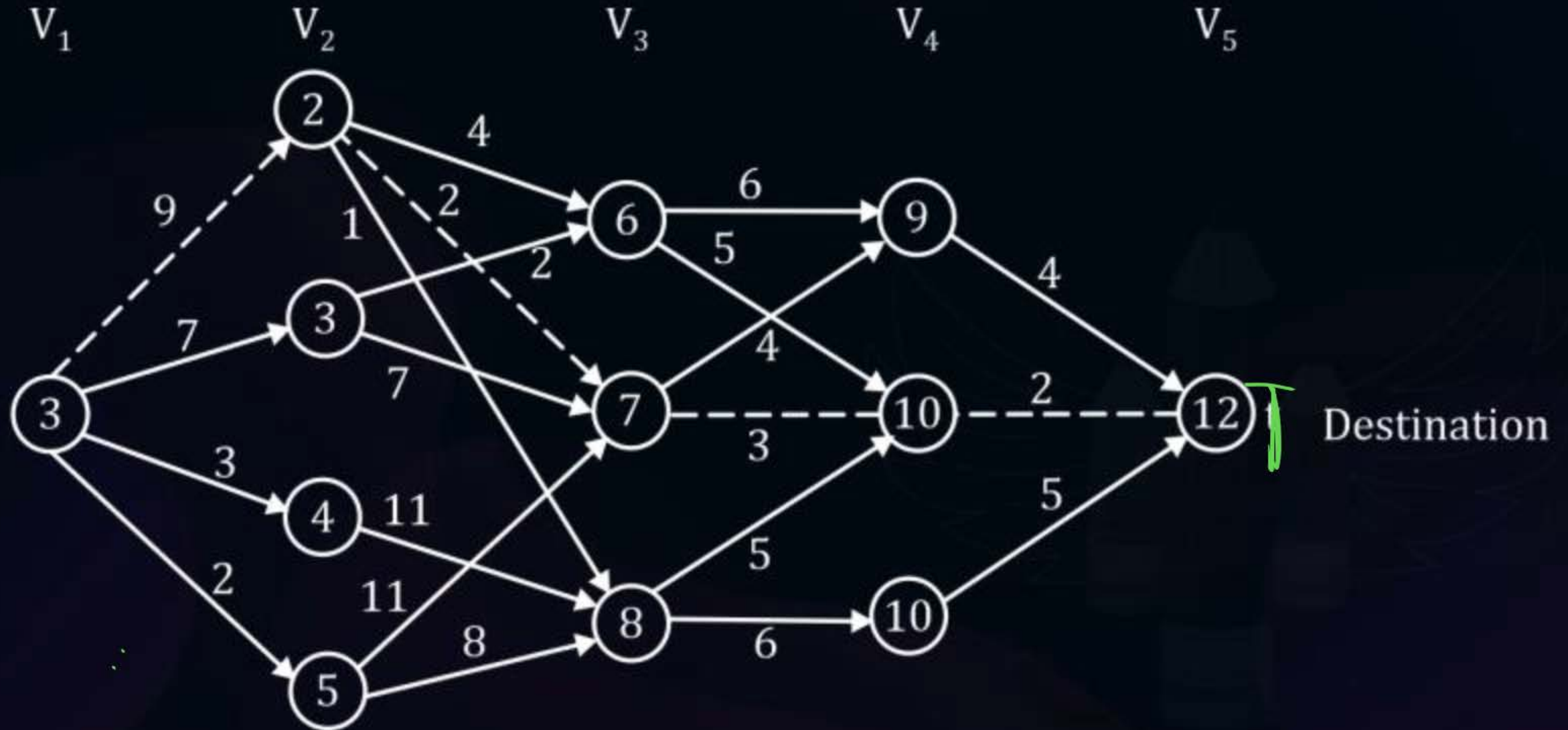
MSG

Stages

Greedy

$S \rightarrow T$

Source
S



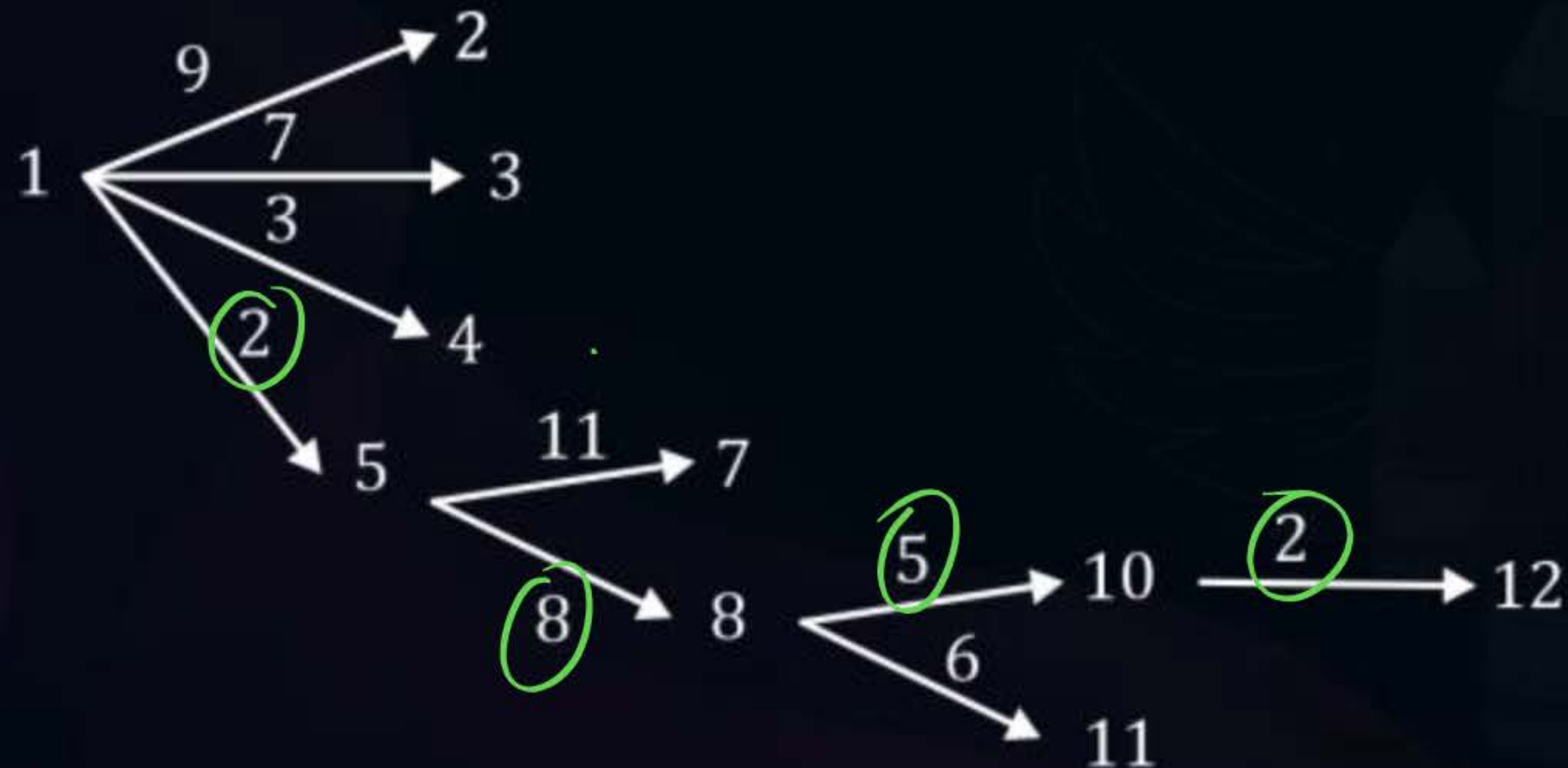


Topic : Dynamic Programming

(s)1 $\rightarrow$ 12(t) using greedy approach.

Greedy approach :- 1 $\rightarrow$ 5 $\rightarrow$ 8 $\rightarrow$ 10 $\rightarrow$ 12

$$2 + 8 + 5 + 2 = \underline{\underline{17}} \text{ \{not optimal\}}$$





Topic : Dynamic Programming

But is 17 optimal /min path cost?

$\langle 1 \rightarrow 2 \rightarrow 7 \rightarrow 10 \rightarrow 12 \rangle$

$$9 + 2 + 3 + 2 = 16$$

$\langle 1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \rightarrow 12 \rangle$

$$7 + 2 + 5 + 2 = 16$$

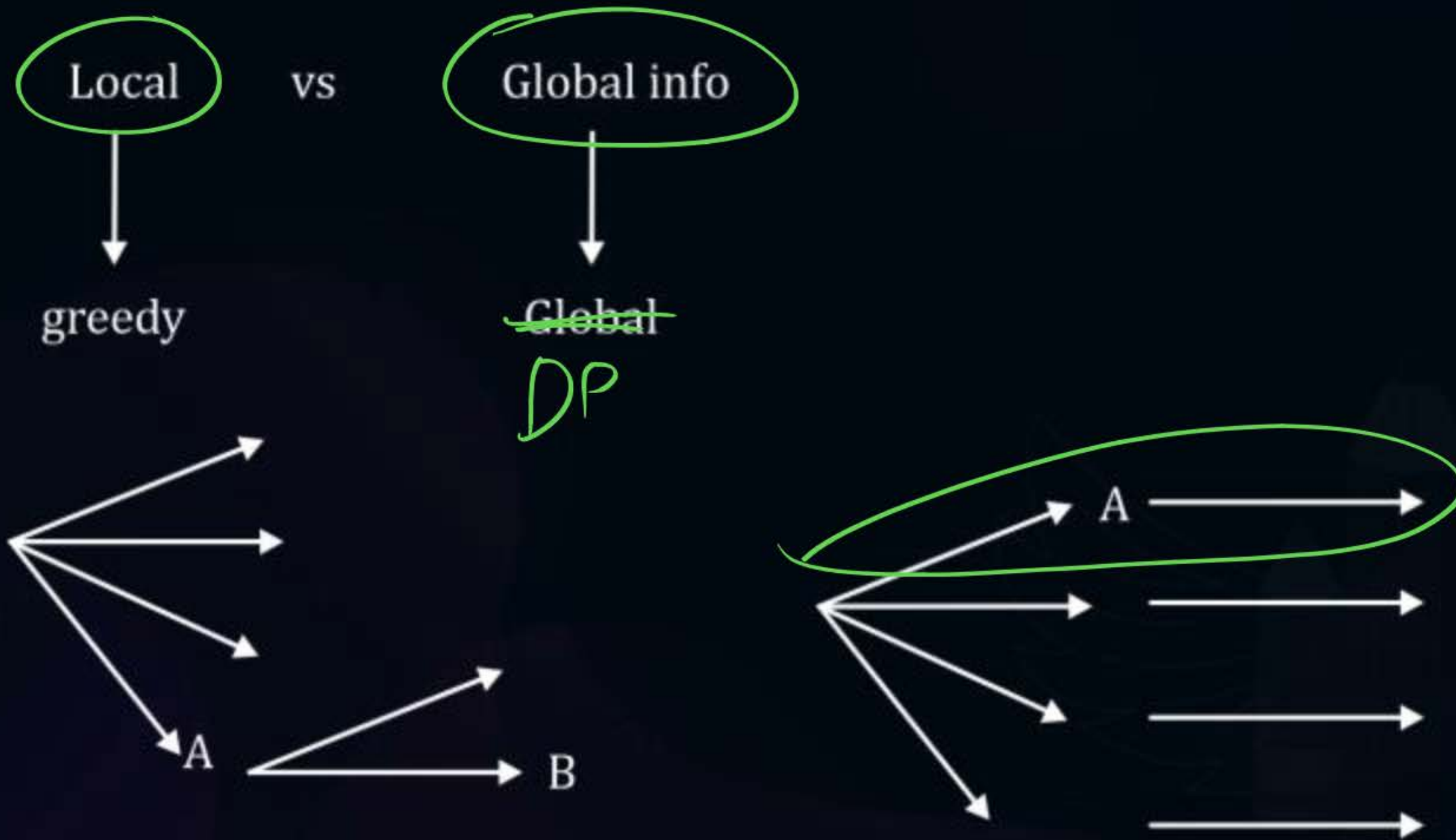
Actual minimum cost
paths from $1 \rightarrow 12$

Hence, greedy approach of SPSP failed to give optimal cost here.

MSG



Topic : Dynamic Programming





Topic : Dynamic Programming: (DP)

DP vs greedy

1. 0/1 Knapsack
2. Coin change problem
3. Multi-stage graph

Greedy → Local optimisation

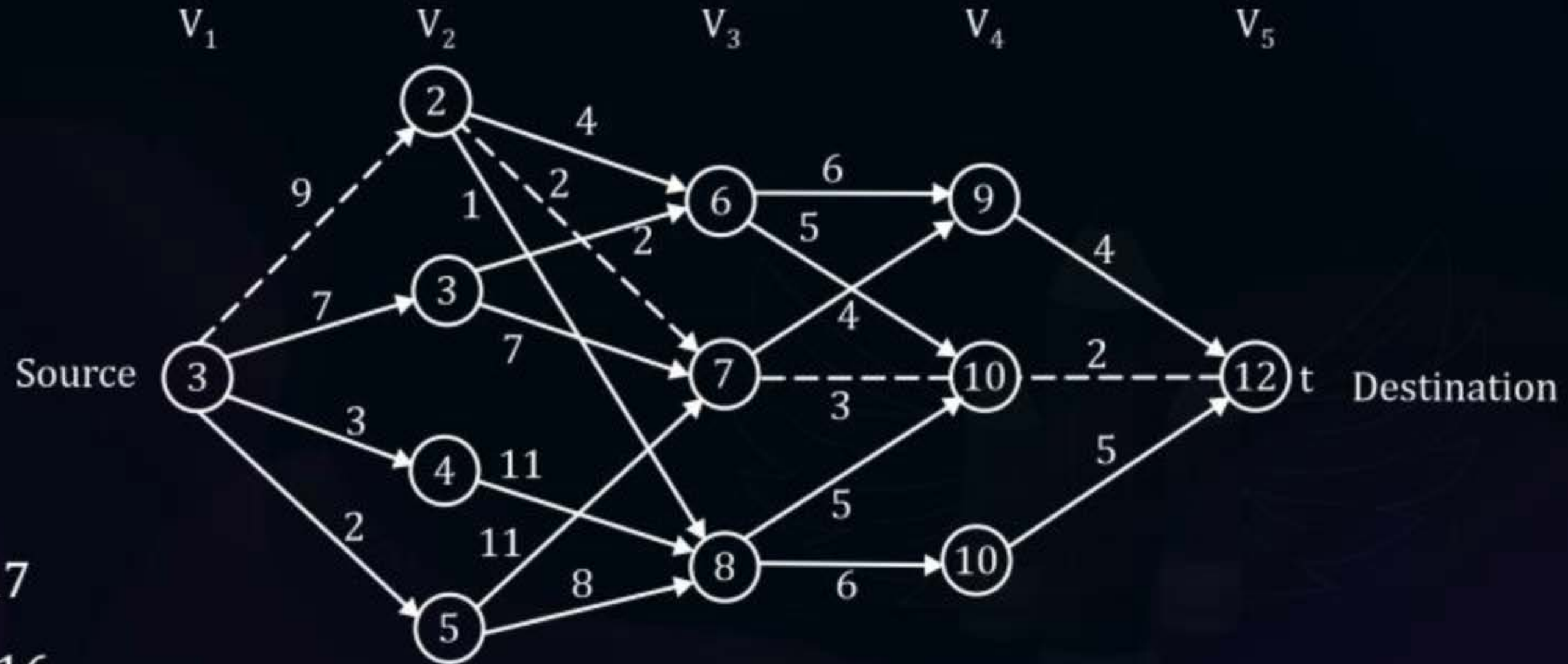
DP → Global optimisation *greedy*

DP gives optimal solution but *greedy* not give optimal solution.



Topic : Dynamic Programming: (DP)

Multi-stage -graph



Greedy $\rightarrow 17$

Optimal $\rightarrow 16$

$$2 + 11 + 3 + 2 = 17$$



Topic : Dynamic Programming: (DP)

P → Path 1 → C1

Path 2 → C2

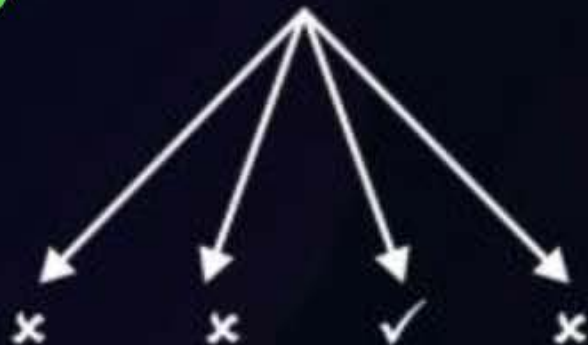
Path 3 → C3

Path 4 → C4

Path 5 → C5

Path n → Cn

DP 0 → Brut force



Min → Optimal solution

Enumeration or Brute force



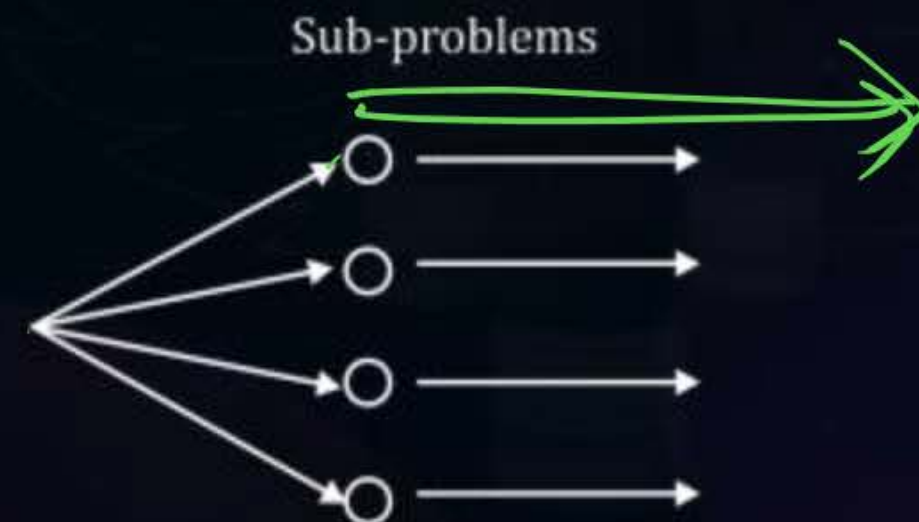
Topic : Dynamic Programming: (DP)

- One way of solving problem in which it is not possible to make sequence of decisions in step-wise manner leading to optimal solutions, is to enumerate all decision sequences (Brute force) & then pick up the best solution (optimal).
- But the drawback with brute force/ Enumeration is excessive Time/Space requirements
- **Dynamic Programming (DP)** based on Enumeration often tries to reduce the amount of enumeration by curtailing those decision sequence from where there is no possibility of getting optimal solution. (That's how it may bring down the time complexity)
STOPPING
- **In Dynamic programming** these set of optimal decisions are made by applying



Topic : Dynamic Programming: (DP)

- Principle of Optimality: states that whatever the initial state & decision are the remaining sequence of decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.
- Essential difference between Greedy method & Dynamic programming (DP) is that Greedy method always generates only one decision sequence. Whereas in DP enumeration many decision sequences can be generated.





Topic : Dynamic Programming: (DP)

V.V. imp

- Another important feature of D.P is that optimal solutions of the sub problems are retained (cached/ stored in a table) to avoid recomputing their values.
(Invariably this feature also leads to saving of time)





Topic : Dynamic Programming: (DP)

- (i) **Splitting of original problems into Subproblems:** Be able to split the original problem into subproblems in a recursive manner (so that the Subproblems can be further divided into sub-subproblems). This process of splitting should continue till the Subproblems becomes small.
- (ii) **Subproblems Optimality (Optimal Substructure):** An optimal solution to the problem must result from optimal solutions to the subproblems with combine operation
- (iii) **Overlapping Subproblems:** Many subproblems themselves contain common sub-subproblems. Therefore, it is desirable to solve the small problem & store/cache their results, so that they can be used in other subproblems.

V.IMP



Topic : Dynamic Programming: (DP)

Fibonacci Number:

Fib series: 0,1,1,2,3,5,8,13,21,34.

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2), \underline{n > 1}$$

$$\text{Fib}(n) = 1, n = 1$$

$$\text{Fib}(n) = 0, n = 0$$



Topic : Dynamic Programming: (DP)

Normal Recursive Implementation of Fib (n):

```
Algo Fib (int n)
{
    If ( $n \leq 1$ ) return (n);
    else
    {
        return (Fib (n-1)+ Fib (n-2));
    }
}
```



Topic : Dynamic Programming: (DP)

Time complexity Recurrence

$$T(n) = C, n = 1$$

$$T(n) = T(n-1) + T(n-2) + a, \underline{n > 1}$$

We know,

$$T(n-2) < T(n-1)$$

$$T(n) = T(n-1) + T(n-1) + a, n > 1$$

$$T(n) = 2T(n-1) + a, n > 1$$

$$T(n-1) = 2T(n-2) + a$$



Topic : Dynamic Programming: (DP)

$$\begin{aligned}T(n) &= 2[2T(n-2) + a] + a \\&= 2^2 T(n-2) + 2a + a \dots (2)\end{aligned}$$

$$T(n-2) = 2T(n-3) + a$$

$$\begin{aligned}T(n) &= 2^2 (2T(n-3) + a) + 2a + a \\&= 2^3 T(n-3) + (2^2 + a \cdot 2^1 + 2^0 a)\end{aligned}$$

General Term

$$\begin{aligned}T(n) &= 2^k T(n-k) + (2^k - 1) * a \\n-k &= 1 \rightarrow k = (n-1)\end{aligned}$$



Topic : Dynamic Programming: (DP)

$$T(n) = 2^{(n-1)} * T(1) + (2^{(n-1)} - 1) * a$$

$$= c * \frac{2^n}{2} + \left(\frac{2^n}{2} - 1 \right) * a$$

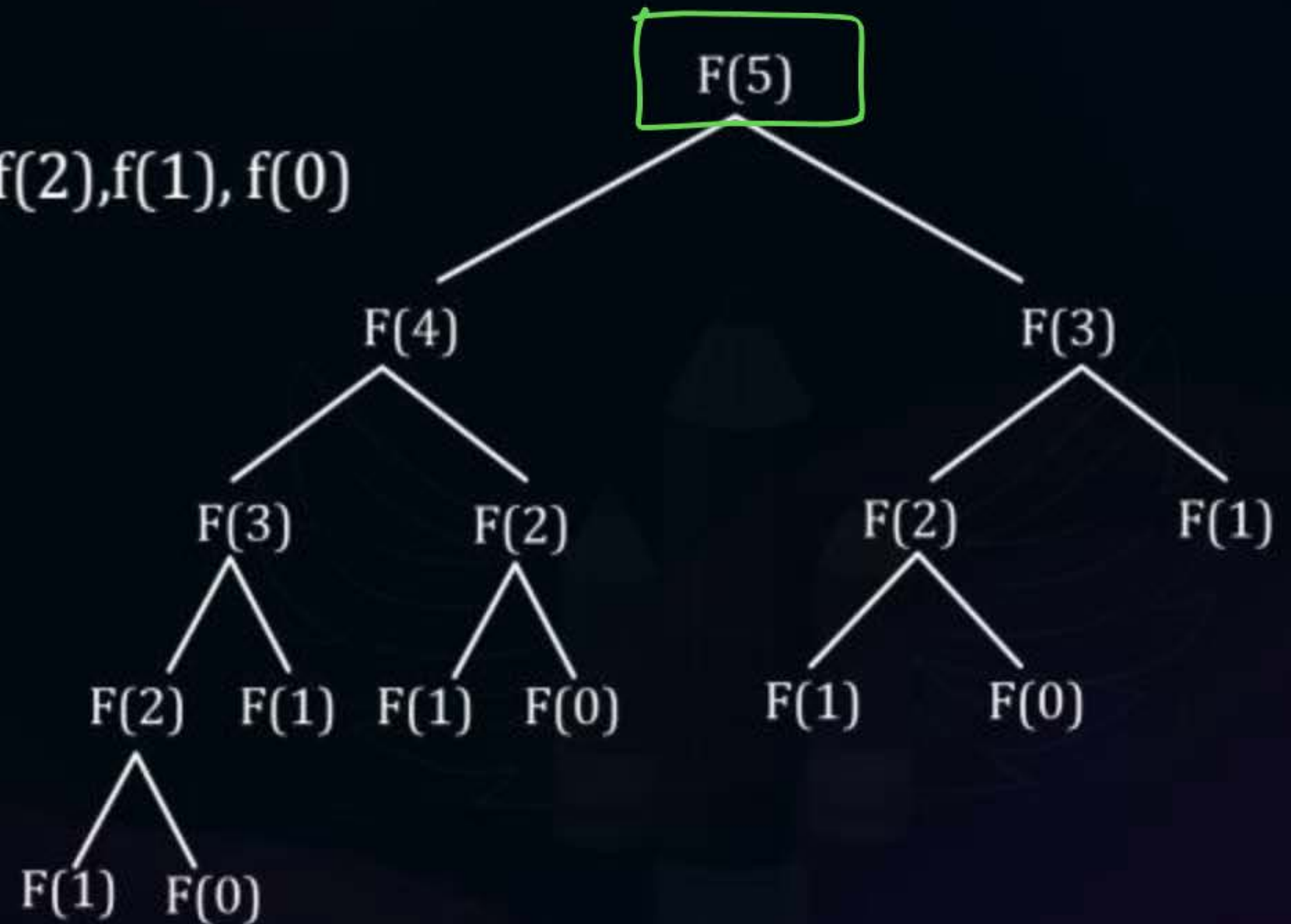
$$T(n) \rightarrow O(2^n)$$





Normal Recursive Approach: Tree of function calls

unique Sub-problems $\rightarrow F(4), f(3), f(2), f(1), f(0)$

$$\approx O(q^n)$$




Topic : Dynamic Programming: (DP)

Top-down-Memoized implantation of Fib (n)

Algo memiFb(n)

{

if (m [n] is undefined)

{

if (n <=1) result = n;

else

result = memFib (n-1) + memFib(n-2);

m[n] = result; //memorizing (caching)

}

return (m[n]);

}



Topic : Dynamic Programming: (DP)

Top-Down code walkthrough:

#Q. Fib (5) = ?

0 → 5

0 1 2 3 4 5

M:

0	1	1	2	3	5
---	---	---	---	---	---

↑
↑
?

memFib(5): res: $3 + 2 = 5 \rightarrow m[5] = 5$

memFib(4): res: $2 + 1 = 3 \rightarrow m[4] = 3$

memFib(3): res: $1 + 1 = 2 \rightarrow m[3] = 2$

memFib(2): res: $1 + 0 = 1 \rightarrow m[2] = 1$

memFib(1): $m[1] = 1$

memFib(0): $m[0] = 0$

memFib(0): 1

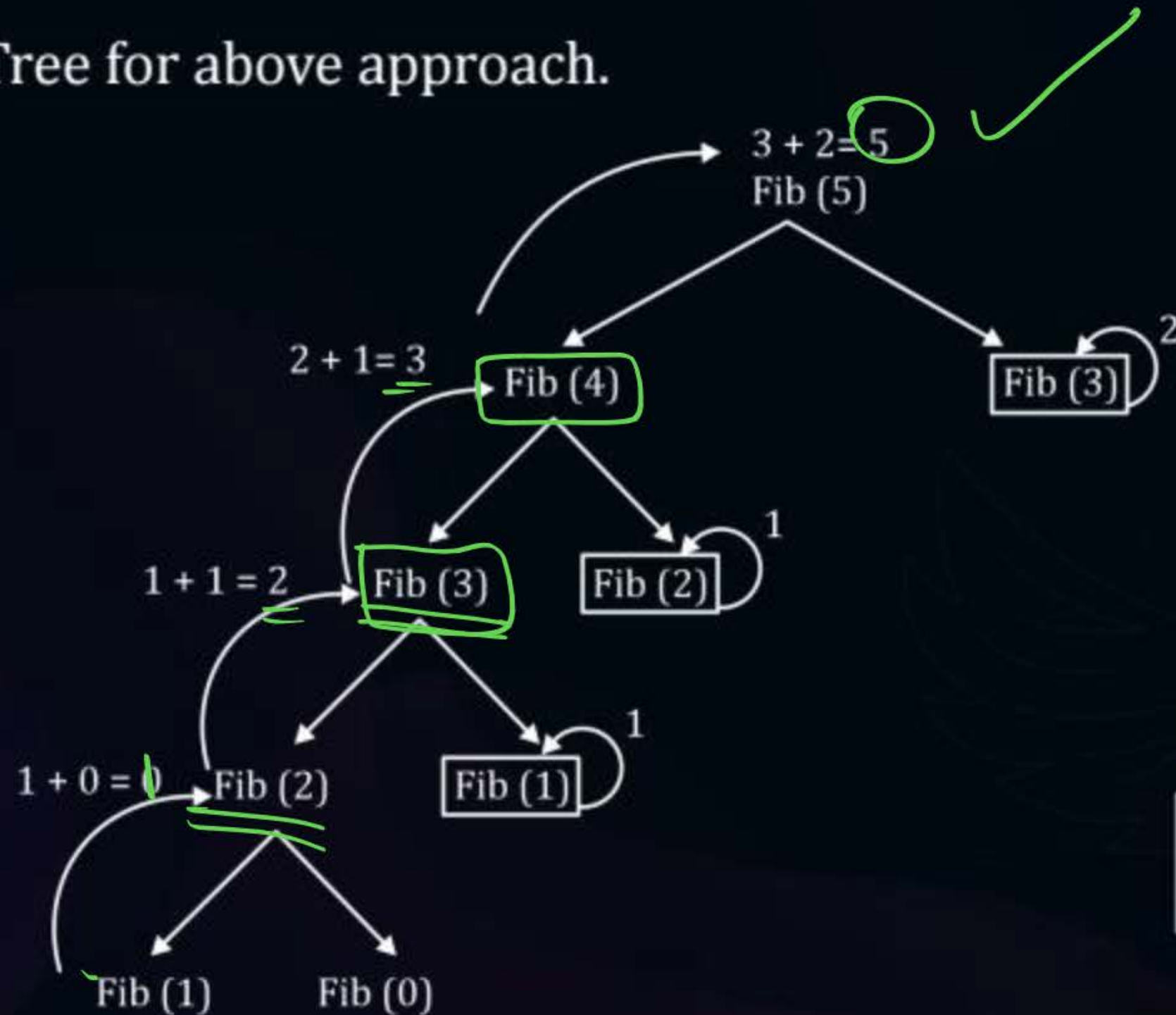
memFib(2): $m[2] = 1$

memFib(3): $m[3] = 2$



Topic : Dynamic Programming: (DP)

Function call Tree for above approach.



TC $\rightarrow O(n)$

$O(2^n)$ $\rightarrow$ $O(n)$



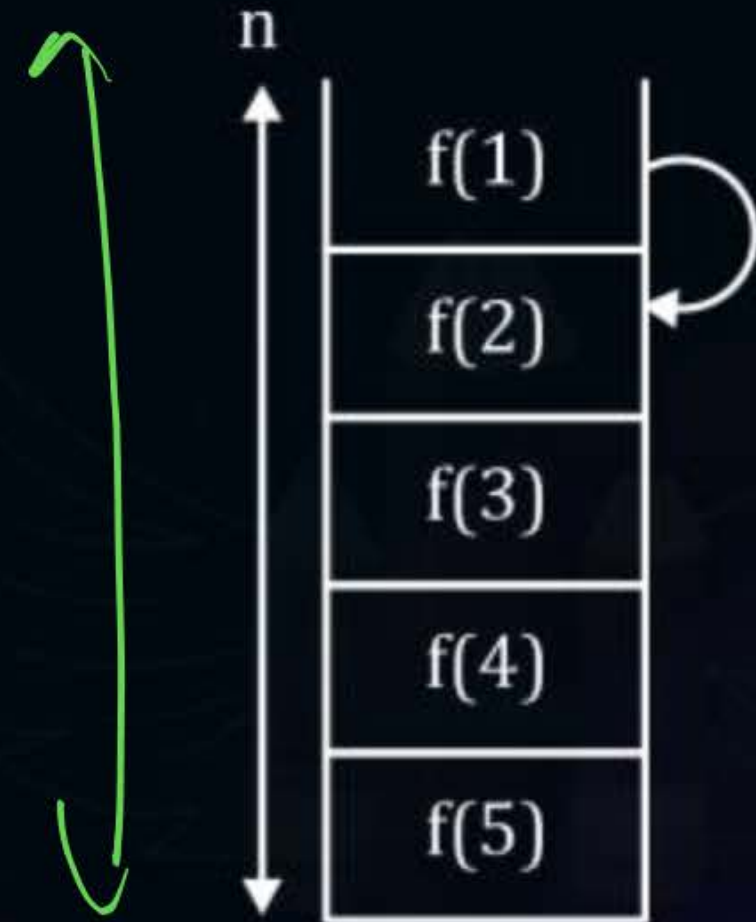
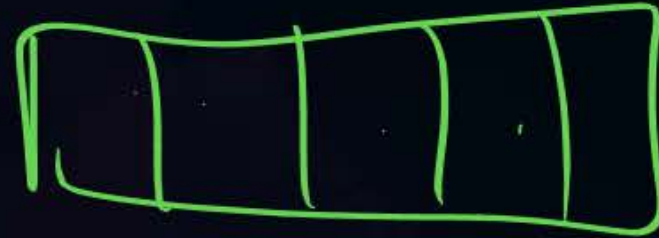
Topic : Dynamic Programming: (DP)

Space Complexity

- Recursion stack $\rightarrow \underline{O(n)}$ {Stack size}
- Auxiliary array $\rightarrow \underline{O(n)}$

Overall space $\rightarrow O(n + n) = O(n)$ ✓

Complexity





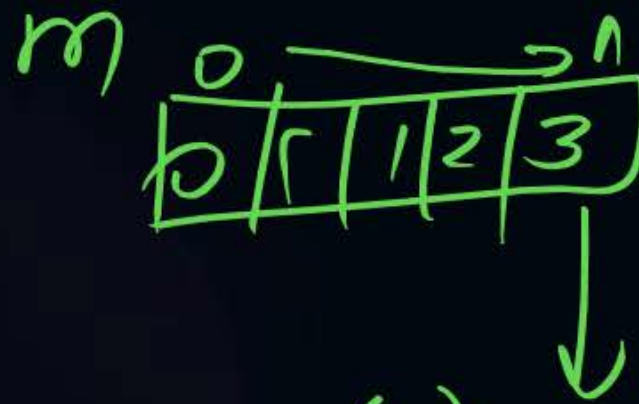
Topic : Dynamic Programming: (DP)

Bottom-up approach of D.P. for Fib (n)

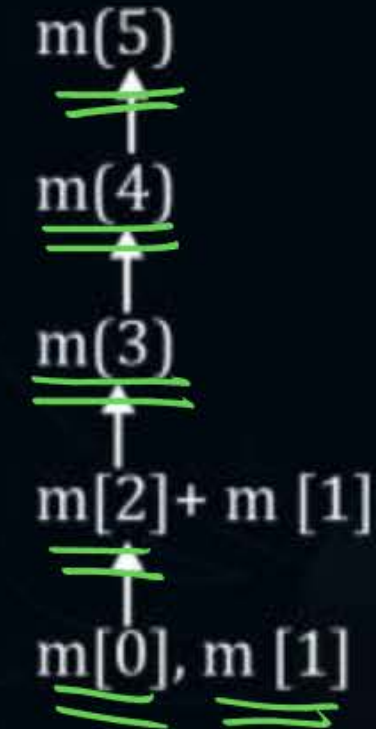
Appr 2: Tabulation (Iterative)

Algo memFib (n)

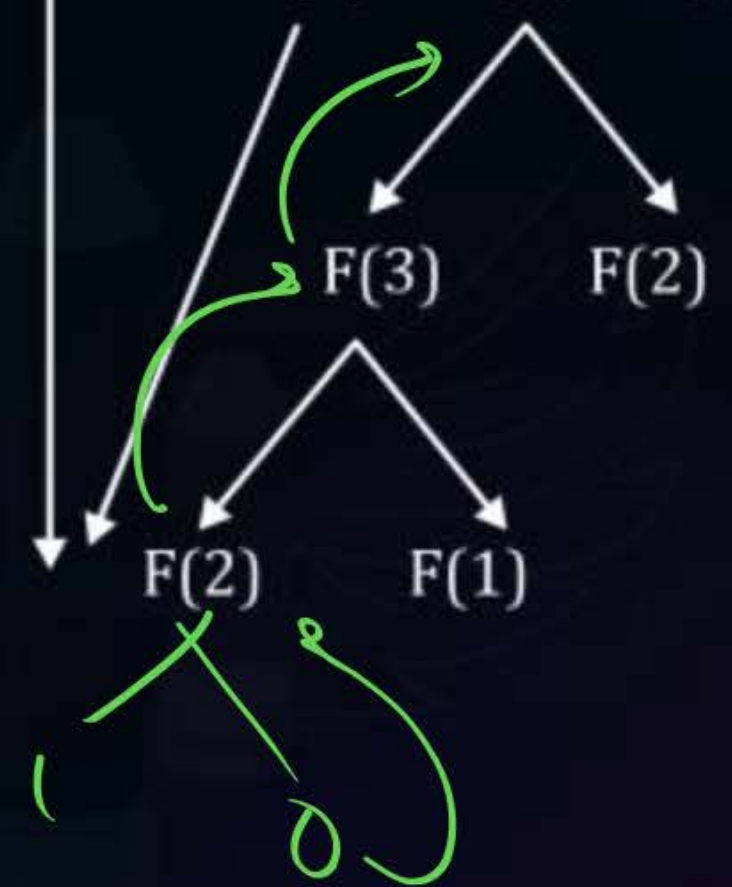
```
{  
    M[0] = 0  
    M[1] = 1  
    For i = 2 to n  
    {  
        M[i] = M[i-1] + M[i-2]  
    }  
    return M[n]  
}
```



Bottom-up



To - Down
F(5) → F(4) ↑ F(3)





Topic : Dynamic Programming: (DP)

Eg.1. **Fib(5)**

Do run

0	1	2	3	4	5
0	1	1	2	3	5

$$\underline{i=2}, m[2] = m[1] + m[0] = 1 + 0 = 1$$

$$\underline{i=3}, m[3] = m[2] + m[1] = 1 + 1 = 2$$

$$\underline{i=4}, m[4] = m[3] + m[2] = 2 + 1 = 3$$

$$O(2^n) \rightarrow O(n)$$

$$TC \rightarrow O(n)$$

$$SC \rightarrow O(n)$$



Topic : Dynamic Programming: (DP)

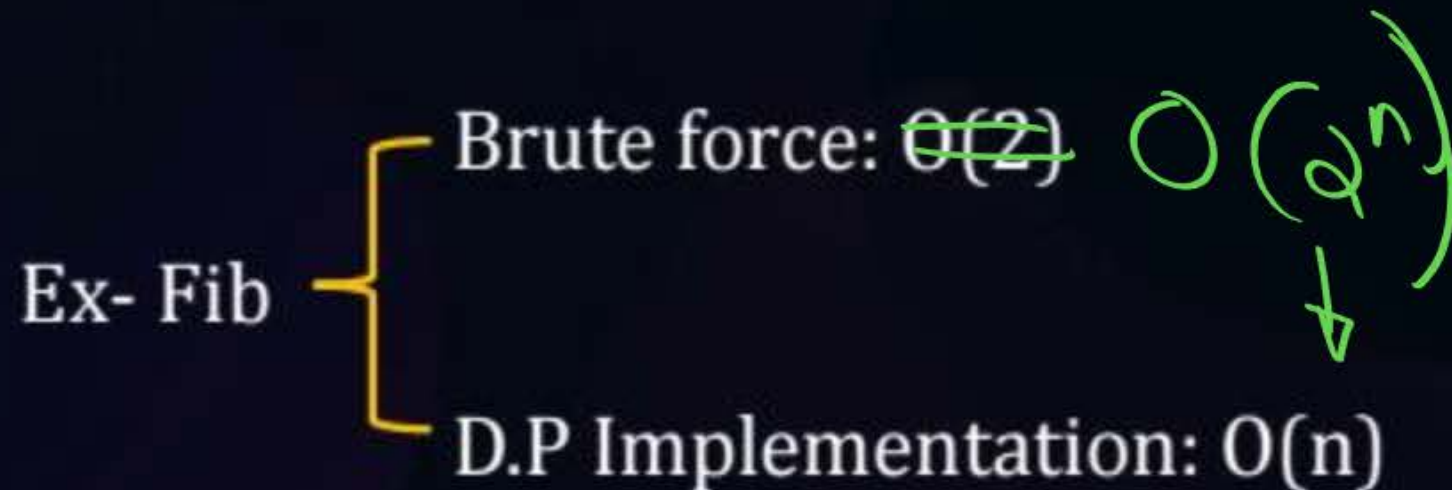
Dynamic Programming vs Greedy Method vs Divide & Conquer:

- In all methods the problem is divided into subproblem;
- **Greedy Method:** Building up of the solution to the problem is done in step-wise manner (incrementally) by applying local options only (local optimality).
- **Divide & conquer:** Breaking up a problem into separate problems (independent), then solve each subproblem separately (i.e. independently) & combine the solution of subproblems to get the solution of original problem.
- **Dynamic Programming:** Breaking up of a problem into a series of overlapping subproblems & building up solution of larger & larger subproblems.



Topic : Dynamic Programming: (DP)

- Unlike Divide and Conquer, D.P typically involves solving all subproblems, rather than a small portion of subproblem.
- D.P tends to solve each subproblem only once since the results of the subproblems are stored, which are used later again when required. This is going to reduce the computation drastically. (In most case, the complexity)





Topic : Dynamic Programming: (DP)



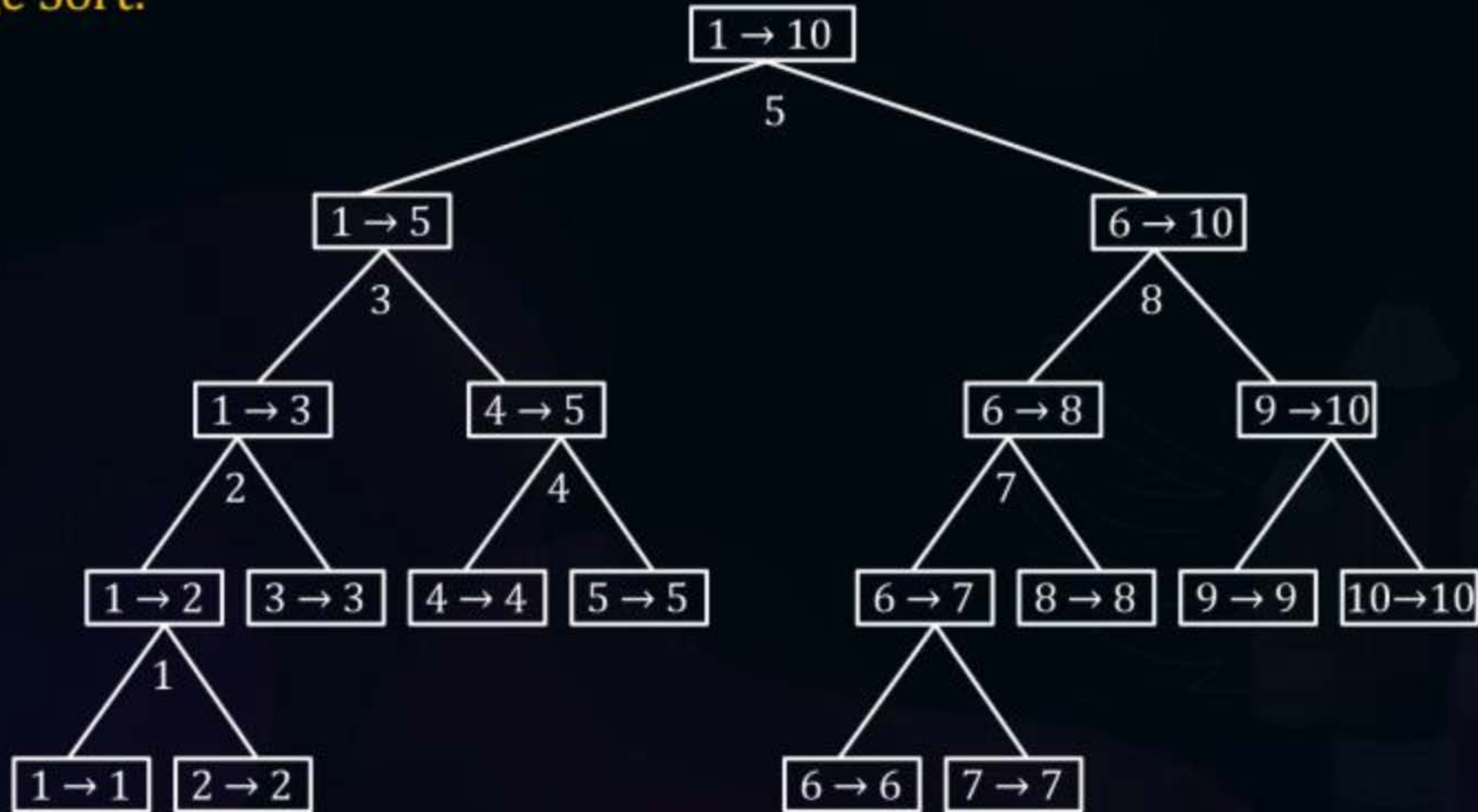
Imp. Diff. D_nC and DP:

1. DnC: independent sub-problems ^{Solve} ~~some~~ separately
2. DP: Overlapping Sub- problems



Topic : Dynamic Programming: (DP)

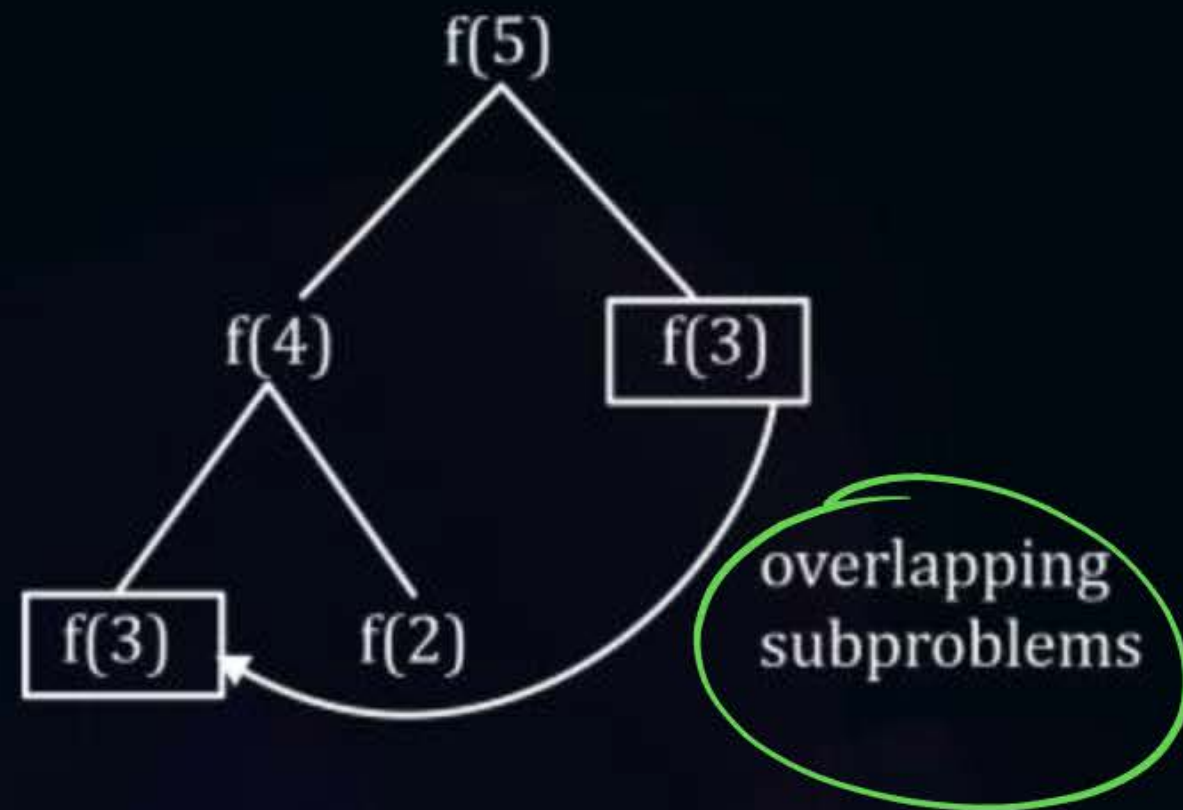
Merge Sort:





Topic : Dynamic Programming: (DP)

DP $\rightarrow$ Fibonacci



Application of DP:



2 mins Summary



Topic

Topic

Topic

Topic

Topic

DP



THANK - YOU